

Abuso de sudo

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

sudo permite a un usuario ejecutar comandos concretos como otro (normalmente root). Si tu usuario tiene permitido ejecutar un binario que además puede lanzar una shell, leer/escribir ficheros o ejecutar código, ese permiso puntual se convierte en root total. Otra cara del privesc-modelo.

El admin concede sudo sobre un binario pensando que solo hace su función, pero muchos binarios tienen funciones secundarias (un editor puede abrir una shell, less puede ejecutar comandos). GTFOBins lista para cada uno como escapar. Además fallos como `env_keep` o rutas relativas amplían el abuso.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Es lo PRIMERO que se mira en privesc Linux por su alta rentabilidad. En cuanto tengas una shell estable (`tty-stabilization`), lanza `sudo -l`.

Cómo identificarla

- `sudo -l` muestra entradas '(root) NOPASSWD: /ruta/bin' (ni siquiera piden password).
- El binario permitido aparece en GTFOBins bajo 'sudo'.
- Ves `env_keep+=LD_PRELOAD` o rutas relativas: vectores extra de abuso.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Ejecuta `sudo -l` y busca cada binario listado en GTFOBins bajo 'sudo'.

Evidencia esperada: Líneas '(root) NOPASSWD: /ruta/binario'. Tras aplicar la receta de GTFOBins, un prompt # con `uid=0`.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

`sudo -l`

Objetivo y contexto: Enumera exactamente que comandos puedes correr como otro usuario. Es el vector mas rapido y comun; muchas rooms se resuelven aqui sin buscar mas.

Resultado esperado: Entradas '(root) NOPASSWD: ...'. Cada binario permitido -> GTFOBins. Si pide password y no la tienes, este vector queda en pausa.

Capa 2: prueba documentada

`sudo /usr/bin/find . -exec /bin/sh \; -quit`

Objetivo y contexto: Ejemplo tipico de GTFOBins: si puedes sudo find, su flag -exec lanza una shell heredando el privilegio root de sudo.

Resultado esperado: Prompt # y id con uid=0. Cambia find por el binario concreto que te permita tu sudo -l; la idea (forzar una shell) es la misma.

Anatomía de los payloads

La primera prueba es `sudo -l`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Lineas '(root) NOPASSWD: /ruta/binario'. Tras aplicar la receta de GTFOBins, un prompt # con uid=0.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Ejecuta `sudo -l` para ver que tienes permitido y si es sin password.
- Busca el binario permitido en GTFOBins, seccion 'sudo'.
- Aplica la linea de escape (suele forzar una shell o ejecutar un comando como root).
- Confirma con id; si eres root, recoge la flag y anota el vector.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- `sudo -l` pide password y no la tienes.
- El binario permitido no aparece en GTFOBins y no tiene funcion de escape conocida (editor sin !shell, etc).

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

sudo -l te dice que puedes ejecutar como root; muchos binarios permiten saltar de ahí a una shell root. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (sudo-abuse)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.